

JC4003 Natural Language Processing

Lecture 8: Word Semantics and Word Embeddings

Dr Xiao Li xiao.li@abdn.ac.uk

Dr Ruizhe Li ruizhe.li@abdn.ac.uk

Dr Siwei Liu siwei.liu@abdn.ac.uk

Outline

Lexical Semantics (i.e. meanings of words)

Word Embedding (i.e. vector semantics)

Context-based word semantic representation

Word to vector (word2vec)

Bert

Lexical Semantics

Lexical Semantics

How to represent the meaning of a word (i.e., lexical semantics) is a shared topic of study in linguistics and machine learning. This is also a kind of **Knowledge Representation**.

Traditionally:

- In classical NLP applications, we only represent a word as a **string of letters**, or **an index** in a vocabulary list.
- In traditional philosophy, the meaning of words is represented by just spelling the word **with small capital letters** (e.g. use **LIFE** to represent the meaning of word 'life').

Polysemous Words and Synonyms

On one hand, some words (polysemous words) have **multiple meanings**, e.g.:

- mouse* (N) ← lemma
- any of numerous small rodents ... ← sense 1
 - a hand-operated device that controls a cursor ... ← sense 2

However, in a sentence, a polysemous word can only express one word sense.

On the other hand, some words have the exactly **similar meanings** (i.e. the synonyms), e.g.:

water / H₂O, couch / sofa, automobile / car, ...

NB: there are probably no examples of perfect synonymy.

Words and Semantics

From these two examples, it is evident that although words convey meaning, they are not the meaning itself.

When we want to express a particular sense, we use words to refer to that sense. Once the listener receives the intended meaning, the word itself becomes irrelevant.

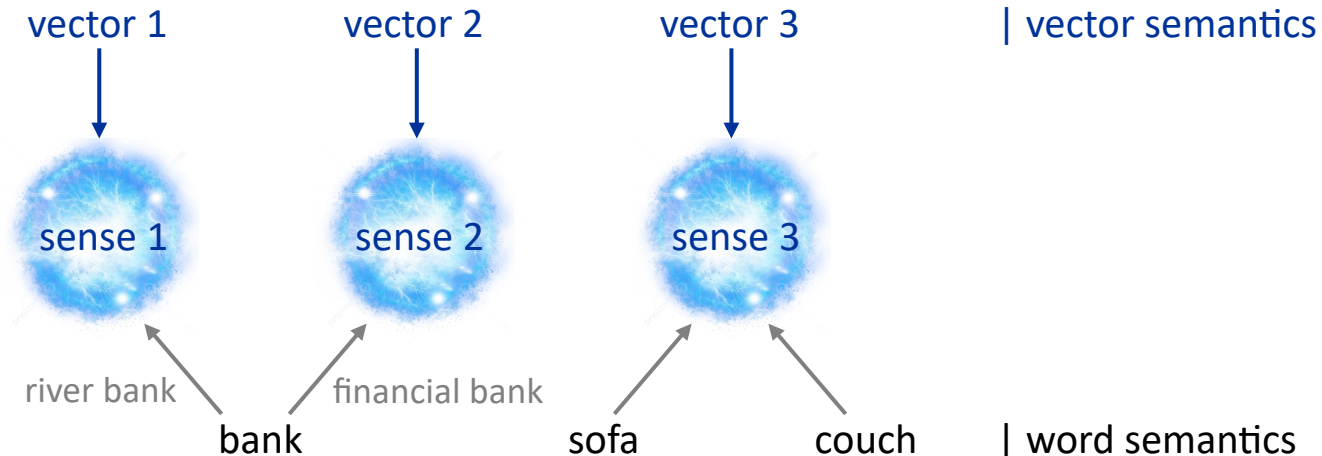
筌者所以在鱼，得鱼而忘筌
蹄者所以在兔，得兔而忘蹄
言者所以在意，得意而忘言

——《庄子·外物》

Vector Semantics

Vector semantics is [the standard way to represent word meaning](#) in NLP, helping us model many of the aspects of word meaning we saw in the previous section.

Vector semantics uses [vector](#) to represent sense of words instead of word itself.



Vector Semantics

Defining sense vectors (or word vectors) is primarily for **computation**. Words themselves cannot be directly used in calculations, but vectors can.

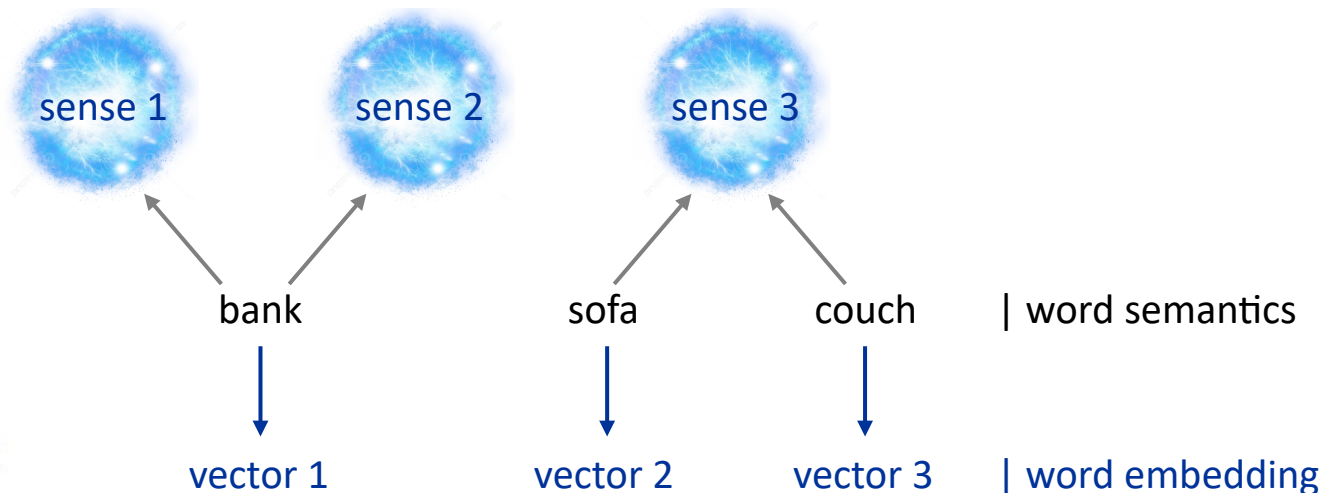
For example, the vectors for "king," "queen," "man," and "woman" should satisfy the following relationship:

$$v_{king} - v_{man} + v_{woman} = v_{queen}$$

Word Embedding

Vector semantics typically represents the **sense** of a word, not just the word itself.

However, representing a sense is challenging, so some works settle for **representing sense of words with vectors instead**. This is called **word embedding**. It is simpler although it is often affected by the multiple senses of polysemous words.



Context-based Word Semantic Representation

Context-based Word Semantics

Words can be defined by their **environments** (i.e. the **context**, the **words around them**).

Linguists like Joos (1950), Harris (1954), and Firth (1957) had the related idea of **defining the meaning of a word by its distribution in language use**, meaning its neighboring words or grammatical environments.

Zellig Harris (1954): If A and B have almost identical environments, we say that they are synonyms.

Example

Suppose you don't know *ongchoi*. But you see these sentences:

- *Ongchoi* is delicious **sautéed with garlic**.
- *Ongchoi* is superb **over rice**
- *Ongchoi* **leaves** with salty sauces

And you've also seen these:

- Spinach **sautéed with garlic over rice**
- Chard stems and leaves are **delicious**
- Collard greens and other **salty** leafy greens

We could conclude that *ongchoi* is a **leafy green** like spinach, chard, or collard greens.



Spinach
菠菜



Chard leaves
甜菜叶



Collard
羽衣甘蓝

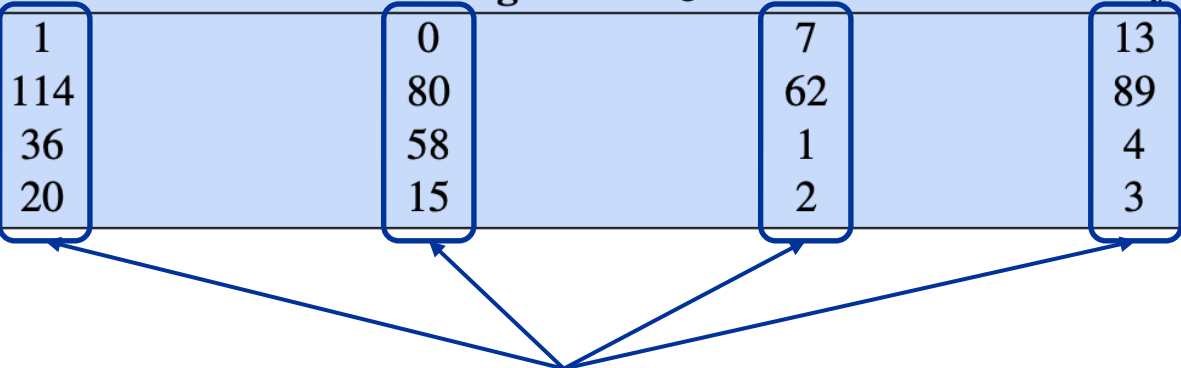
Term-document Matrix

Term-document Matrix

A simplified approach is to consider each text in the corpus as an indivisible context and then examine the [distribution of words across the entire corpus](#).

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

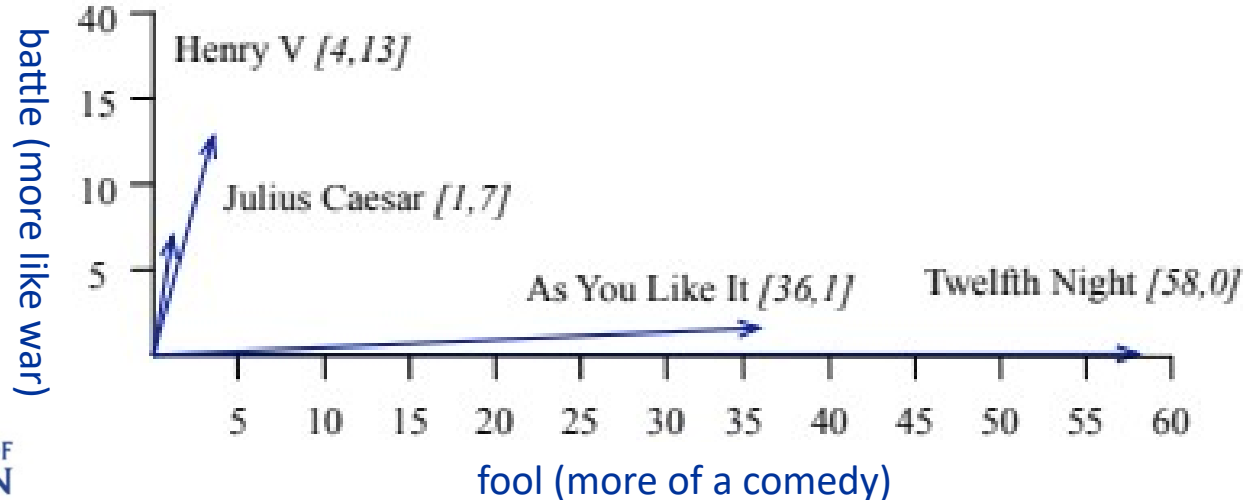
document vectors



Visualising Document Vectors

A spatial visualisation of the document vectors for the four Shakespeare play documents, showing **just two of the dimensions**, corresponding to the words **battle** and **fool**.

The comedies have high values for the fool dimension and low values for the battle dimension.



Term-document Matrix

Vector semantics can also [represent the meaning of words](#). We do this by associating each word with a word vector (a row vector rather than a column vector), hence with different dimensions, e.g.:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

word vectors

Cosine Similarity for Words

When we represent words as vectors, we can **measure the similarity** of their meanings using vector similarity, specifically cosine similarity.

$$\text{similar}(w_1, w_2) = \cos(\angle w_1 w_2) = \frac{w_1 \cdot w_2}{|w_1||w_2|}$$

Since cosine similarity ranges from **-1 to 1**, when two vectors have

1. **opposite** meanings, their similarity approaches to **-1**;
2. **unrelated** meanings, the similarity is roughly **0**;
3. **similar** meanings, the similarity approaches to **+1**.

Limitations of the Term-Document Matrix

As the Term-Document Matrix uses raw frequency as vector parameters, it gives high weight to frequently occurring words like "a," "the," and similar terms.

However, these high-frequency words often lack substantial semantic meaning, which can significantly disrupt vector semantics.

TF-IDF

Inverse Document Frequency (IDF)

Inverse Document Frequency (IDF) is the component of TF-IDF that measures how important a word is across the entire corpus.

While word frequency measures how often a word appears in a specific document, IDF helps to balance this **by giving less importance to words that are common** across many documents and more importance to words that are rare or specific to a few documents.

- **Common Words:** Words that appear in many documents (like "the", "is", "and") are often not very informative for distinguishing between documents. These words are given lower weights by the IDF component.
- **Rare Words:** Words that **appear in only a few documents** are considered more informative or indicative of the content of those documents. These words receive higher weights in the IDF calculation.

Inverse Document Frequency (IDF)

The IDF for a term t (e.g. a word) is defined based on logarithm:

$$\text{IDF}(t) = \log_2 \frac{N_d}{DF(t)}$$

where N_d is the total number of documents in the corpus, and $DF(t)$ is the number of documents in which the term t appears (i.e. the document frequency).

According to the definition of IDF:

- common words are assigned very small weights,
- rare words are given greater attention.

TF-IDF

Therefore, for each vector constructed using word frequency, we can multiply each frequency by the IDF to mitigate the impact of excessively high weights assigned to common words. We call this the **TFIDF** or **TF-IDF**.

$$\text{TFIDF}(t) = \text{TF}(t, d) \cdot \text{IDF}(t) = \text{TF}(t, d) \cdot \log_2 \frac{N_d}{\text{DF}(t)}$$

where $\text{TF}(t, d)$ denotes the **term (word) frequency** in document d .

PS: in some areas, people use $\log_{10}$ or $\log_e$ rather than $\log_2$.

TF-IDF

Here are the raw counts in the Shakespeare term-document matrix, and the TF-IDF weighted version of the same matrix.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Raw freq



	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

TF-IDF

Limitations TF-IDF-based Vectors

When constructing word vectors using TF-IDF, several potential issues may arise:

- **High Dimensionality:** TF-IDF vectors tend to be **very long** because the vector length is equal to the size of the vocabulary. In large corpora, where the vocabulary size can be in the tens or hundreds of thousands, this results in extremely high-dimensional vectors. High dimensionality can lead to increased computational costs and challenges in managing and processing the data efficiently.
- **Sparsity:** TF-IDF vectors are often sparse, meaning that they **contain many zeros**. This sparsity occurs because most documents only contain a small subset of the total vocabulary. As a result, the majority of elements in each vector are zeros, which can make the vectors less effective in capturing the nuanced relationships between words and can also contribute to inefficiencies in storage and computation.

Word2Vec

Word2Vec

Word2Vec is a **neural network-based technique** that transforms words into continuous vectors, capturing their semantic meanings.

Developed by Google in 2013, Word2Vec uses two main approaches—Continuous Bag of Words (CBOW) and Skip-Gram—to learn word representations from large text corpora.

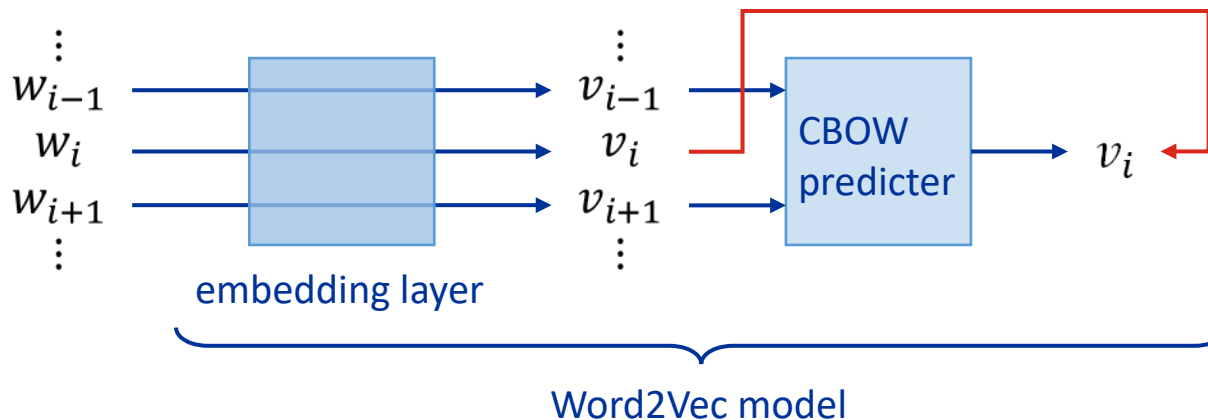
Compared to TF-IDF-based word vectors, word2vec produces **dense and short** vectors (100 and 300 dimensions).

Word2Vec

Word2Vec uses a classifier to obtain word vectors, rather than a counting method.

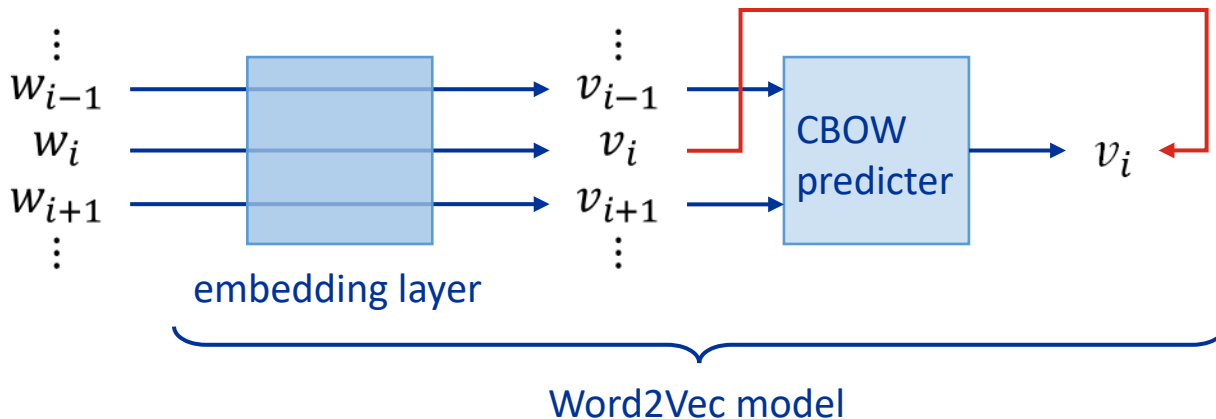
It establishes a classifier to predict the relationship between each word in the corpus (referred to as the target word w_i) and its surrounding words (i.e. its context).

After training, Word2Vec uses classifier's embedding layer as the word vectors (i.e. v_i , rather than using the classifier's prediction output).



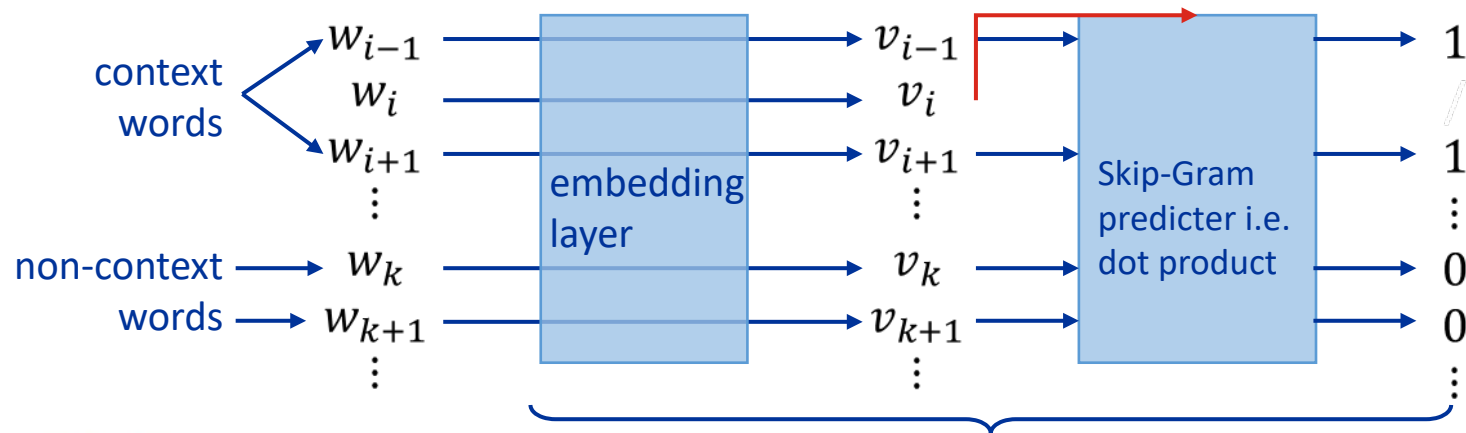
Word2Vec (cont.)

- The **embedding layer** is a simple matrix. It assigns each word a vector. Given a word, the embedding layer just selects and outputs the vector assigned to the word.
- CBOW predictor is a simple dense neural network.
- Word2Vec pioneered the self-supervised training: **self-supervision is a learning method where a model generates its own labels from the input data.**



Word2Vec (cont.)

Word2Vec also has another training method called [Skip-Gram](#). Unlike CBOW, Skip-Gram's self-supervised task is different: given a target word w_i paired with a candidate context word c it will return the probability that if c is a real context word.



Word2Vec

Contextual Similarity: Word2Vec captures semantic relationships between words by placing similar words closer together in the vector space. It effectively captures context-based word meanings.

Efficient Training: It is computationally efficient, able to process large corpora and generate high-quality word embeddings relatively quickly.

Pre-trained Model: you can also download the pre-trained vectors:

- Word2vec (Mikolov et al.) <https://code.google.com/archive/p/word2vec/>

GloVe

GloVe (Global Vectors for Word Representation) is another popular word embedding technique, which is similar to Word2Vec.

GloVe derives its word embeddings based on the **word co-occurrence probabilities**, capturing **global statistical relationships** between words in the corpus.

GloVe directly predicts the logarithm of co-occurrence probabilities using word vectors, i.e.:

$$\mathcal{L} = \sum_{w_i \in Voc} \sum_{w_j \in Voc} (v_i^T \cdot v_j - \log_2 X_{ij})$$

where X_{ij} is the **co-occurrence probabilities** of word w_i and w_j ; v_i and v_j are the embedding vector of w_i and w_j . $\mathcal{L}$ is the loss to be minimised.

Visualising Embeddings

Visualizing embeddings is an important goal in helping understand, apply, and improve these models of word meaning.

The simplest way to visualise the meaning of a word embedded is to **list the most similar words** to it by sorting all the word vectors by their cosine similarity. E.g., the 7 closest words to "**frog**" using a particular embeddings computed with the GloVe are:

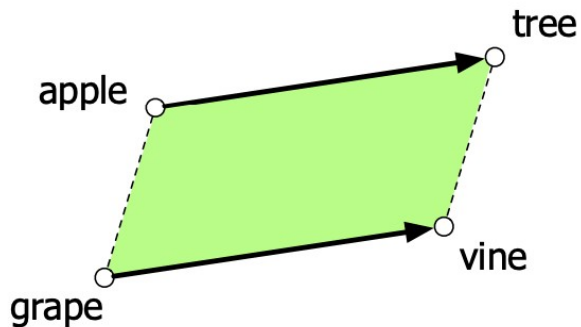
frogs, toad, litoria, leptodactylidae, rana, lizard, eleutherodactylus

Visualising Embeddings

The most common visualization method, however, is to project the high dimensions of a word down into 2 dimensions, e.g.:

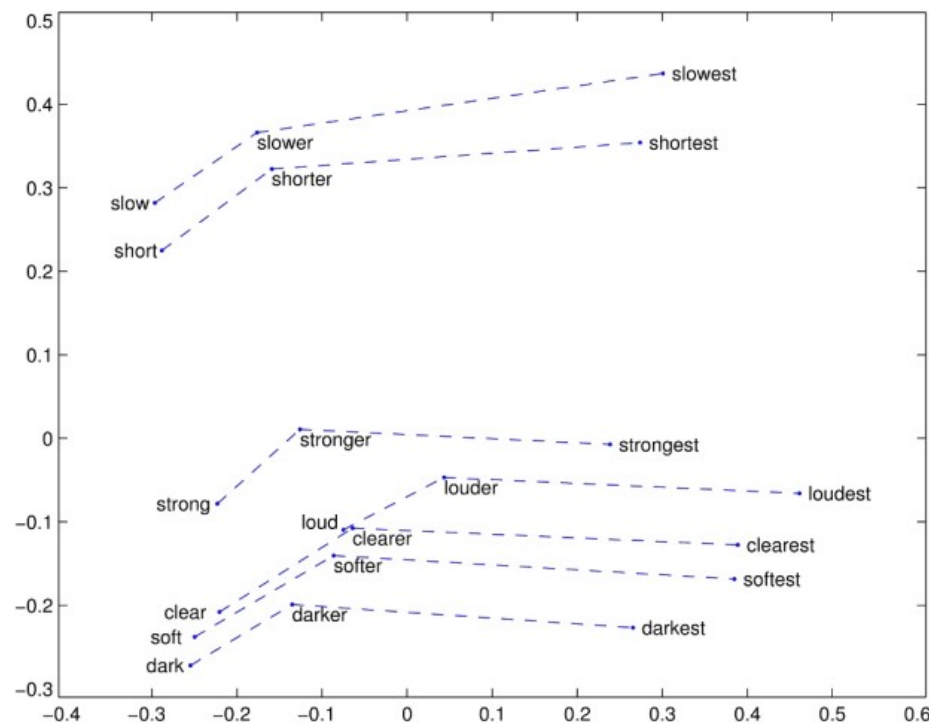
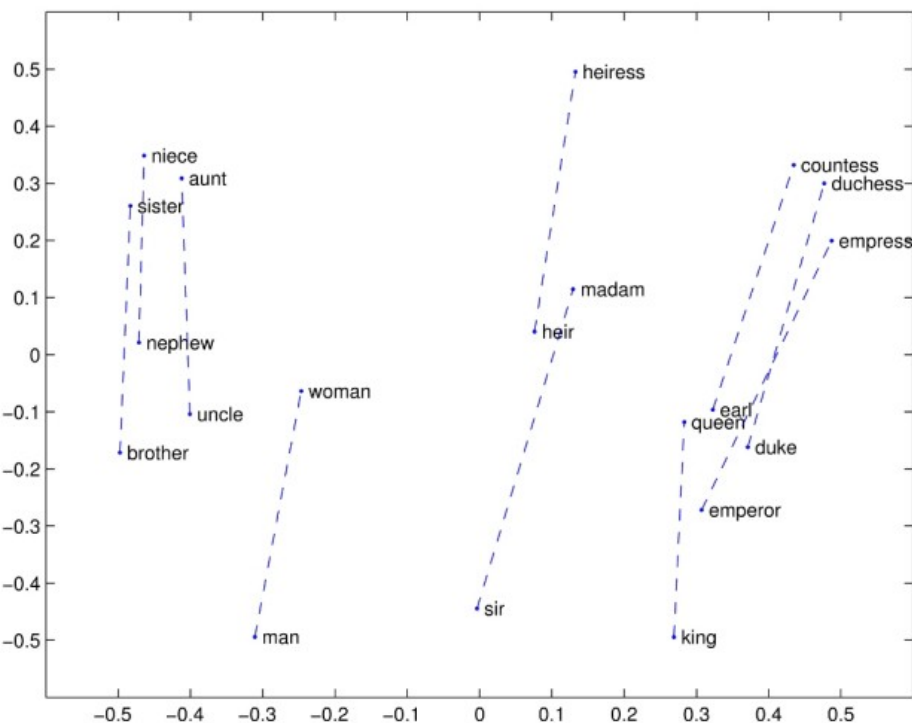
The parallelogram model for analogy problems : the location of $\overrightarrow{vine}$ can be found by subtracting $\overrightarrow{apple}$ from $\overrightarrow{tree}$ and adding $\overrightarrow{grape}$. E.g.

$$\overrightarrow{apple} - \overrightarrow{tree} = \overrightarrow{grape} - \overrightarrow{vine}$$



Visualising Embeddings

The similar visualisation is showing below.



Conclusion

Lexical Semantics: Explored how word meanings are represented, considering polysemous words and synonyms.

Vector Semantics: Introduced the concept of representing word meanings with vectors, capturing relationships between words.

Word Embeddings: Discussed embedding methods like Word2Vec and GloVe, which efficiently encode words into continuous vectors based on context.

TF-IDF: Covered how term frequency-inverse document frequency adjusts word weights by considering word rarity across documents.

Contextual Similarity: Highlighted how embeddings capture semantic relationships, placing similar words closer in vector space.